Types

Système de types

"A type system is a tractable syntactic method for proving the absence of certain program behaviors by classifying phrases according to the kinds of values they compute."

Benjamin Pierce, *Types and Programming Languages*

Système de types

"A type system is a tractable syntactic method for proving the absence of certain program behaviors by classifying phrases according to the kinds of values they compute."

Benjamin Pierce, *Types and Programming Languages*

- Approximation du comportement des programmes

Système de types

"A type system is a tractable syntactic method for proving the absence of certain program behaviors by classifying phrases according to the kinds of values they compute."

Benjamin Pierce, *Types and Programming Languages*

- Approximation du comportement des programmes
- Garantit l'absence, pas la présence

`if true then 5 else` *erreur*

- Dépend du langage

Système de types

"A type system is a tractable syntactic method for proving the absence of certain program behaviors by classifying phrases according to the kinds of values they compute."

Benjamin Pierce, *Types and Programming Languages*

- Approximation du comportement des programmes
- Garantit l'absence, pas la présence

`if true then 5 else` *erreur*

- Dépend du langage
- Équilibre expressivité/utilisation pratique
- Souvent à la compilation (construction de l'AST)

Utilité (en programmation)

"Well typed programs cannot go wrong."

Robin Milner, *A Theory of Type Polymorphism in Programming*

- Détection des erreurs
- Abstraction (modules en OCaml, ...)
- Documentation du code

Noyau OCaml

Types

$$\begin{array}{rll} \tau ::= & \texttt{num} & \text{(type des nombres)} \\ & \texttt{str} & \text{(type des chaînes)} \end{array}$$

Expressions

$$\begin{array}{rll} e ::= & x & \text{(variable)} \\ & n & \text{(nombre littéral)} \\ & s & \text{(chaîne littérale)} \\ & e_1 + e_2 & \text{(addition)} \\ & e_1 \texttt{\^{}} e_2 & \text{(concaténation)} \\ & \texttt{let } x\texttt{=}e_1 \texttt{ in } e_2 & \text{(définition)} \end{array}$$

Terme mal formé : 10 + “hello”

Jugements et règles de typages

- De la forme $\Gamma \vdash e : \tau$
- Environnement de typage $\Gamma, \Delta ::= x_1 : \tau_1, \ldots, x_n : \tau_n$
  - sans répétition de variables : $x_1 \ldots x_n$ deux-à-deux distinctes
  - domaine $\mathrm{dom}(\Gamma) = \{x_1 \ldots x_n\}$
  - $\mathrm{fv}(e) \subseteq \mathrm{dom}(\Gamma)$

Jugements et règles de typages

- De la forme $\Gamma \vdash e : \tau$
- Environnement de typage $\Gamma, \Delta ::= x_1{:}\tau_1, \ldots, x_n{:}\tau_n$
  - sans répétition de variables : $x_1 \ldots x_n$ deux-à-deux distinctes
  - domaine $\mathrm{dom}(\Gamma) = \{x_1 \ldots x_n\}$
  - $\mathrm{fv}(e) \subseteq \mathrm{dom}(\Gamma)$

$$\textsc{Num}\ \frac{}{\Gamma \vdash n : \mathsf{num}} \qquad \textsc{Str}\ \frac{}{\Gamma \vdash s : \mathsf{str}}$$

$$\textsc{Add}\ \frac{\Gamma \vdash e_1 : \mathsf{num} \qquad \Gamma \vdash e_2 : \mathsf{num}}{\Gamma \vdash e_1 + e_2 : \mathsf{num}} \qquad \textsc{Conc}\ \frac{\Gamma \vdash e_1 : \mathsf{str} \qquad \Gamma \vdash e_2 : \mathsf{str}}{\Gamma \vdash e_1 \hat{} e_2 : \mathsf{str}}$$

Règles et dérivation de typage

$$\text{Var}\ \frac{}{\Gamma, x : \tau \vdash x : \tau} \qquad \text{Let}\ \frac{\Gamma \vdash e_1 : \tau_1 \qquad \Gamma, x : \tau_1 \vdash e_2 : \tau_2}{\Gamma \vdash \texttt{let}\ x = e_1\ \texttt{in}\ e_2 : \tau_2}$$

Le typage est dirigé par la syntaxe (décoration de l'ABT)

Règles et dérivation de typage

$$\text{Var}\ \frac{}{\Gamma, x : \tau \vdash x : \tau} \qquad \text{Let}\ \frac{\Gamma \vdash e_1 : \tau_1 \qquad \Gamma, x : \tau_1 \vdash e_2 : \tau_2}{\Gamma \vdash \texttt{let}\ x = e_1\ \texttt{in}\ e_2 : \tau_2}$$

Le typage est dirigé par la syntaxe (décoration de l'ABT)

Exemple : dérivation de typage de `let` x=10 `in` x+1.

Règles et dérivation de typage

$$\text{Var}\ \frac{}{\Gamma, x : \tau \vdash x : \tau} \qquad \text{Let}\ \frac{\Gamma \vdash e_1 : \tau_1 \qquad \Gamma, x : \tau_1 \vdash e_2 : \tau_2}{\Gamma \vdash \texttt{let}\ x = e_1\ \texttt{in}\ e_2 : \tau_2}$$

Le typage est dirigé par la syntaxe (décoration de l'ABT)

Exemple : dérivation de typage de `let` x=10 `in` x+1.

- Est-ce que `let` x="*coucou*" `in` x est typable ?

Règles et dérivation de typage

$$\text{Var}\ \frac{}{\Gamma, x : \tau \vdash x : \tau} \qquad \text{Let}\ \frac{\Gamma \vdash e_1 : \tau_1 \qquad \Gamma, x : \tau_1 \vdash e_2 : \tau_2}{\Gamma \vdash \texttt{let}\ x = e_1\ \texttt{in}\ e_2 : \tau_2}$$

Le typage est dirigé par la syntaxe (décoration de l'ABT)

Exemple : dérivation de typage de `let` x=10 `in` x+1.

- Est-ce que `let` x="*coucou*" `in` x est typable ? oui
- Est-ce que `let` x=10 `in` x ^ "*salut*" est typable ?

Règles et dérivation de typage

$$\text{Var}\ \frac{}{\Gamma, x : \tau \vdash x : \tau} \qquad \text{Let}\ \frac{\Gamma \vdash e_1 : \tau_1 \qquad \Gamma, x : \tau_1 \vdash e_2 : \tau_2}{\Gamma \vdash \texttt{let}\ x = e_1\ \texttt{in}\ e_2 : \tau_2}$$

Le typage est dirigé par la syntaxe (décoration de l'ABT)

Exemple : dérivation de typage de `let` x=10 `in` x+1.

- Est-ce que `let` x="*coucou*" `in` x est typable ? oui
- Est-ce que `let` x=10 `in` x ^ "*salut*" est typable ? non

Propriétés

Lemme (Unicité du type résultat)

Pour tout e *et* Γ *tel que* $dom(\Gamma) \supseteq \mathrm{fv}(e)$, *il existe au plus un type* τ *tel que* $\Gamma \vdash e : \tau$.

Démonstration.

Induction sur e. □

Propriétés

Lemme (Unicité du type résultat)

Pour tout e *et* Γ *tel que* $dom(\Gamma) \supseteq \mathrm{fv}(e)$, *il existe au plus un type* τ *tel que* $\Gamma \vdash e : \tau$.

Démonstration.

Induction sur e. □

Lemme (Affaiblissement)

Pour tout e, Γ, τ *tel que* $\Gamma \vdash e : \tau$, *pour tout* x, τ' *tel que* $x \notin \mathrm{dom}(\Gamma)$, *on a* $\Gamma, x : \tau' \vdash e : \tau$.

Démonstration.

Induction sur $\Gamma \vdash e : \tau$. □

Propriétés

Lemme (Substitution)

Pour tout Γ, x, τ_1, e_2, τ_2 *tel que* $\Gamma, x : \tau_1 \vdash e_2 : \tau_2$, *pour tout* e_1 *tel que* $\Gamma \vdash e_1 : \tau_1$, *on a* $\Gamma \vdash e_2\{e_1/x\} : \tau_2$.

Démonstration.

Exercice □

Lemme (Décomposition)

Et réciproquement

Démonstration.

Exercice □

Fonctions d'ordre supérieur

Types

$\tau ::=$	`num`	(type des nombres)
	`func`	(type des fonctions)

Expressions

$e ::=$	x	(variable)
	n	(nombre littéral)
	$e_1 + e_2$	(addition)
	$e_1\ e_2$	(application)
	`fun` $(x{:}\tau) \Rightarrow e$	(abstraction)
	`let` $x{=}e_1$ `in` e_2	(définition)

Fonctions d'ordre supérieur

Types

$\tau ::=$	`num`	(type des nombres)
	`func`	(type des fonctions)

Expressions

$e ::=$	x	(variable)
	n	(nombre littéral)
	$e_1 + e_2$	(addition)
	$e_1\ e_2$	(application)
	`fun` $(x{:}\tau) \Rightarrow e$	(abstraction)
	`let` $x{=}e_1$ `in` e_2	(définition)

Distinguer
`fun` $(x{:}\texttt{num}) \Rightarrow x{+}1$ et
`fun` $(f{:}\texttt{func}) \Rightarrow$ `fun` $(x{:}\texttt{num}) \Rightarrow f\ x$

Fonctions d'ordre supérieur

Types

$\tau ::=$	num	(type des nombres)
	$\tau_1 \to \tau_2$	(type des fonctions)

Expressions

$e ::=$	x	(variable)
	n	(nombre littéral)
	$e_1 + e_2$	(addition)
	$e_1\ e_2$	(application)
	$\texttt{fun}\ (x{:}\tau)\ \Rightarrow\ e$	(abstraction)
	$\texttt{let}\ x{=}e_1\ \texttt{in}\ e_2$	(définition)

Type des fonctions

- $\tau_1 \to \tau_2$: argument τ_1, résultat τ_2
- associatif à droite : $\tau_1 \to \tau_2 \to \tau_3 = \tau_1 \to (\tau_2 \to \tau_3)$

Fun

$$\frac{\Gamma, x : \tau_1 \vdash e : \tau_2}{\Gamma \vdash \texttt{fun}\ (x{:}\tau_1)\ \Rightarrow\ e : \tau_1 \to \tau_2}$$

App

$$\frac{\Gamma \vdash e_1 : \tau_1 \to \tau_2 \qquad \Gamma \vdash e_2 : \tau_1}{\Gamma \vdash e_1\ e_2 : \tau_2}$$

Type des fonctions

- $\tau_1 \to \tau_2$: argument τ_1, résultat τ_2
- associatif à droite : $\tau_1 \to \tau_2 \to \tau_3 = \tau_1 \to (\tau_2 \to \tau_3)$

Fun

$$\frac{\Gamma, x : \tau_1 \vdash e : \tau_2}{\Gamma \vdash \texttt{fun}\ (x{:}\tau_1)\ \Rightarrow\ e : \tau_1 \to \tau_2}$$

App

$$\frac{\Gamma \vdash e_1 : \tau_1 \to \tau_2 \qquad \Gamma \vdash e_2 : \tau_1}{\Gamma \vdash e_1\ e_2 : \tau_2}$$

- Type de $\texttt{fun}\ (x{:}\text{num})\ \Rightarrow\ x{+}1$? $\text{num} \to \text{num}$
- Type de $\texttt{fun}\ (f{:}\text{num} \to \text{num})\ \Rightarrow\ \texttt{fun}\ (x{:}\text{num})\ \Rightarrow\ f\ x$?
  $(\text{num} \to \text{num}) \to \text{num} \to \text{num}$

Propriétés

Unicité, affaiblissement, substitution, décomposition toujours valides

Propriétés

Unicité, affaiblissement, substitution, décomposition toujours valides

Types à la Church vs à la Curry

- à la Church : $\Gamma \vdash \texttt{fun}\ (x{:}\mathsf{num}) \Rightarrow x{+}1 : \mathsf{num} \to \mathsf{num}$
- à la Curry : $\Gamma \vdash \texttt{fun}\ x \Rightarrow x{+}1 : \mathsf{num} \to \mathsf{num}$

Propriétés

Unicité, affaiblissement, substitution, décomposition toujours valides

Types à la Church vs à la Curry

- à la Church : $\Gamma \vdash \texttt{fun}\ (x{:}\mathsf{num}) \Rightarrow x{+}1 : \mathsf{num} \to \mathsf{num}$
- à la Curry : $\Gamma \vdash \texttt{fun}\ x \Rightarrow x{+}1 : \mathsf{num} \to \mathsf{num}$

Est-ce que l'unicité du type résultat est toujours valide dans le langage à la Curry ?